

Today: Last Consistency Paper
Exploiting Nil-Externality - SOSp'21

Next: GFS, Dynamo, Big Table

Final Project

Exploiting Nil-Externality for Fast Replicated Storage



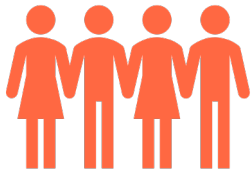
Reproducible

Aishwarya Ganesan, Ramnatthan Alagappan (*VMware Research*)

Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau (*University of Wisconsin – Madison*)

SOSP'21

Apps Depend Upon Distributed Storage



social networking



e-commerce



video streaming



finance



ride sharing



APACHE
ZooKeeper™



mongoDB®



redis



APACHE
HBASE



etcd



cassandra



riak



kafka



Couchbase



CockroachDB

Distributed storage systems



Consistency
avoid unintuitive results



Reliability
data safe despite failures



Performance
efficient access to data



tradeoff between guarantees and performance

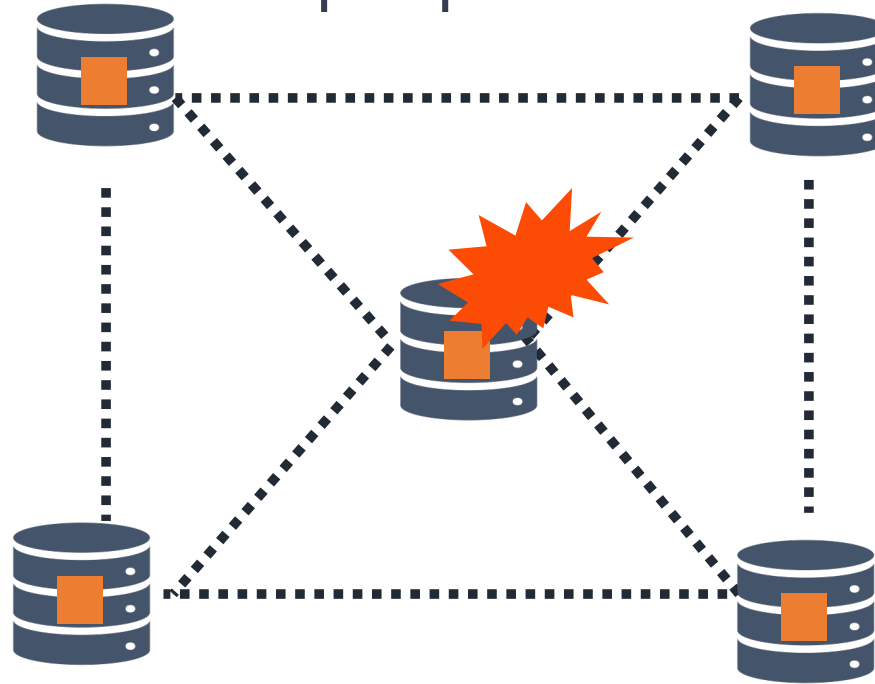
Reliability is Important

Single copy of data



Does not tolerate failures

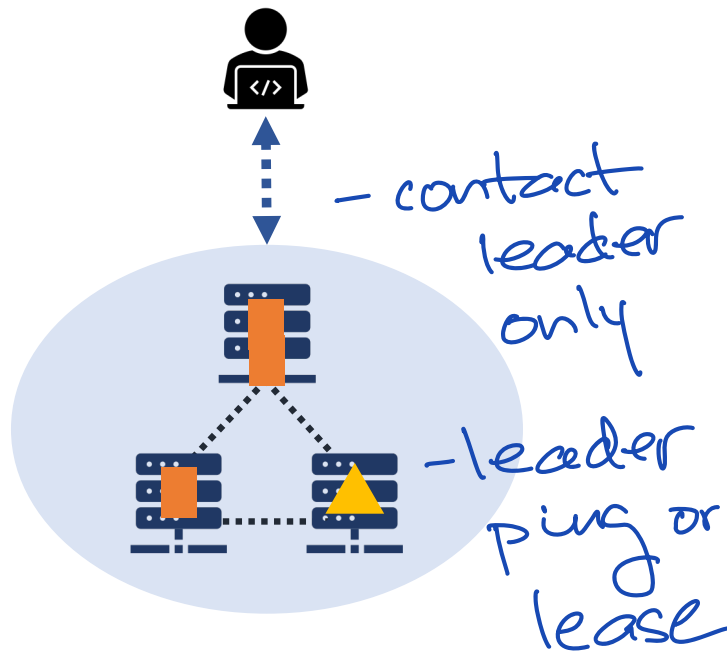
Multiple copies of data



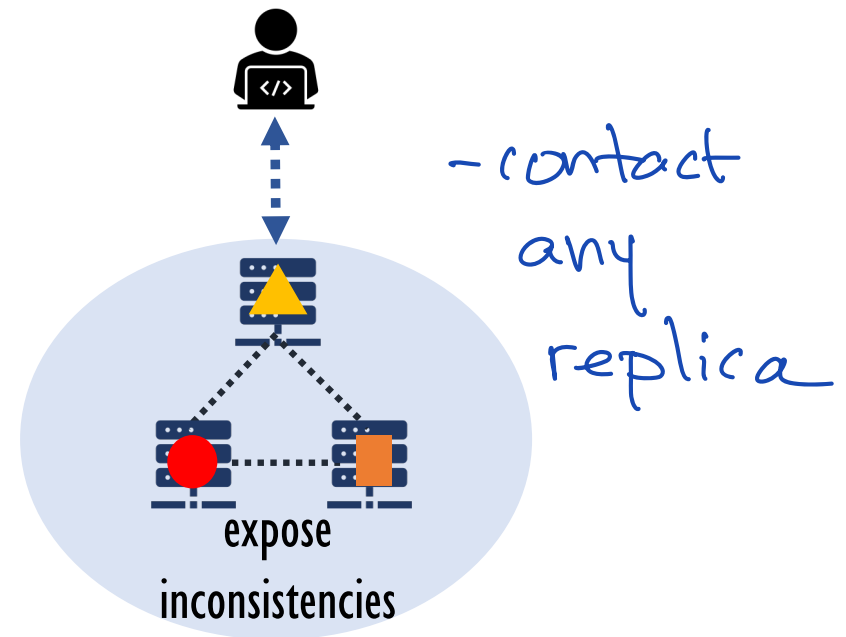
Tolerate failures

Might have worse consistency and/or performance

Handle Failures: Consistency vs. Performance



strong consistency
ease of programming
performs poorly



weak consistency
hard to reason — difficult to program
performs well

act like a
single copy

The Two Worlds



Favor strong consistency over performance



Favor high performance over consistency

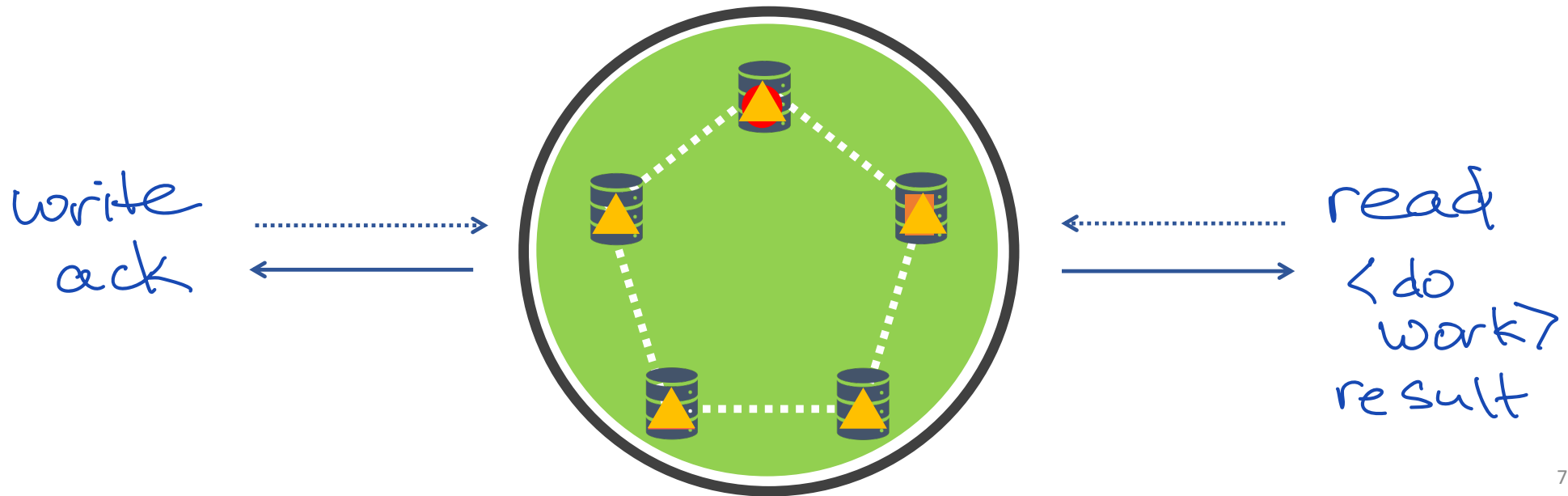


Emerging Approach

High performance → Defer work

Inconsistent → but only **internal**

Strong consistency → Consistency enforced when



Similar to Occult - enforced causal dependencies when needed

Outline

Nilext-aware replication

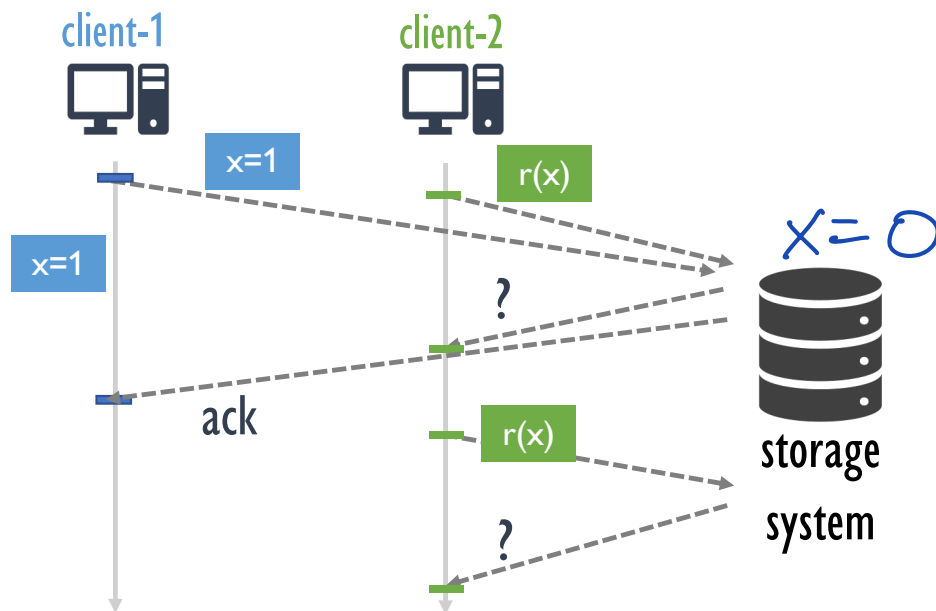
Strongly consistent systems background 

Storage interfaces and nil-externality

Exploiting nil-externality

Skyros design and results

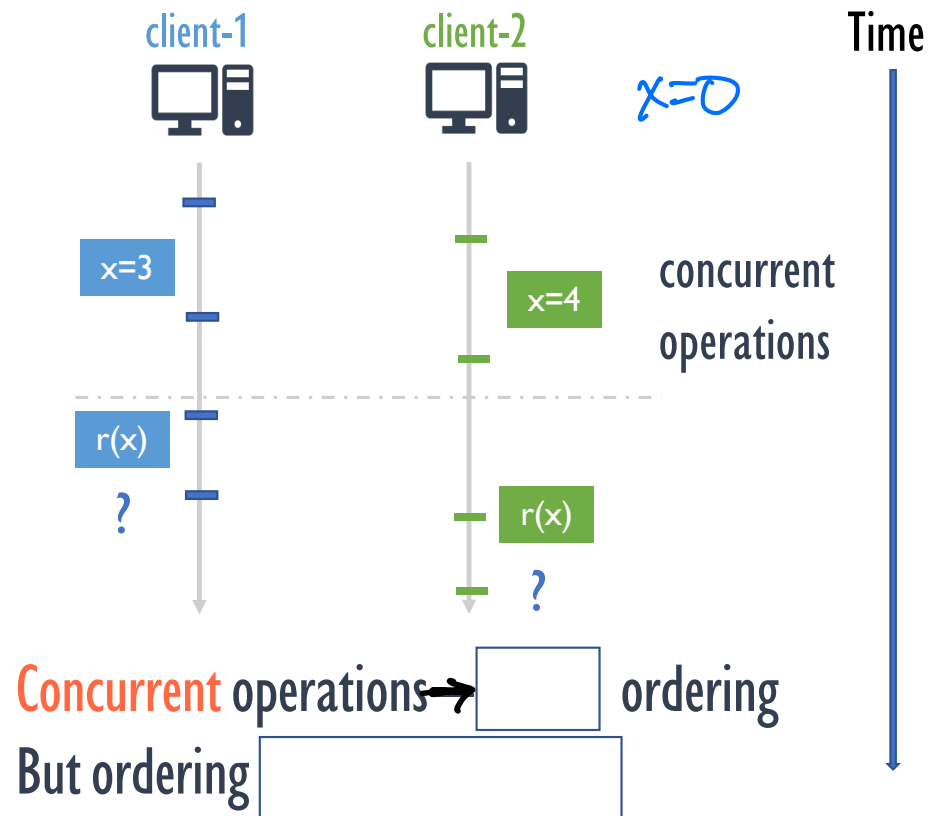
Background: Strong Consistency



Strong consistency (linearizability) respects

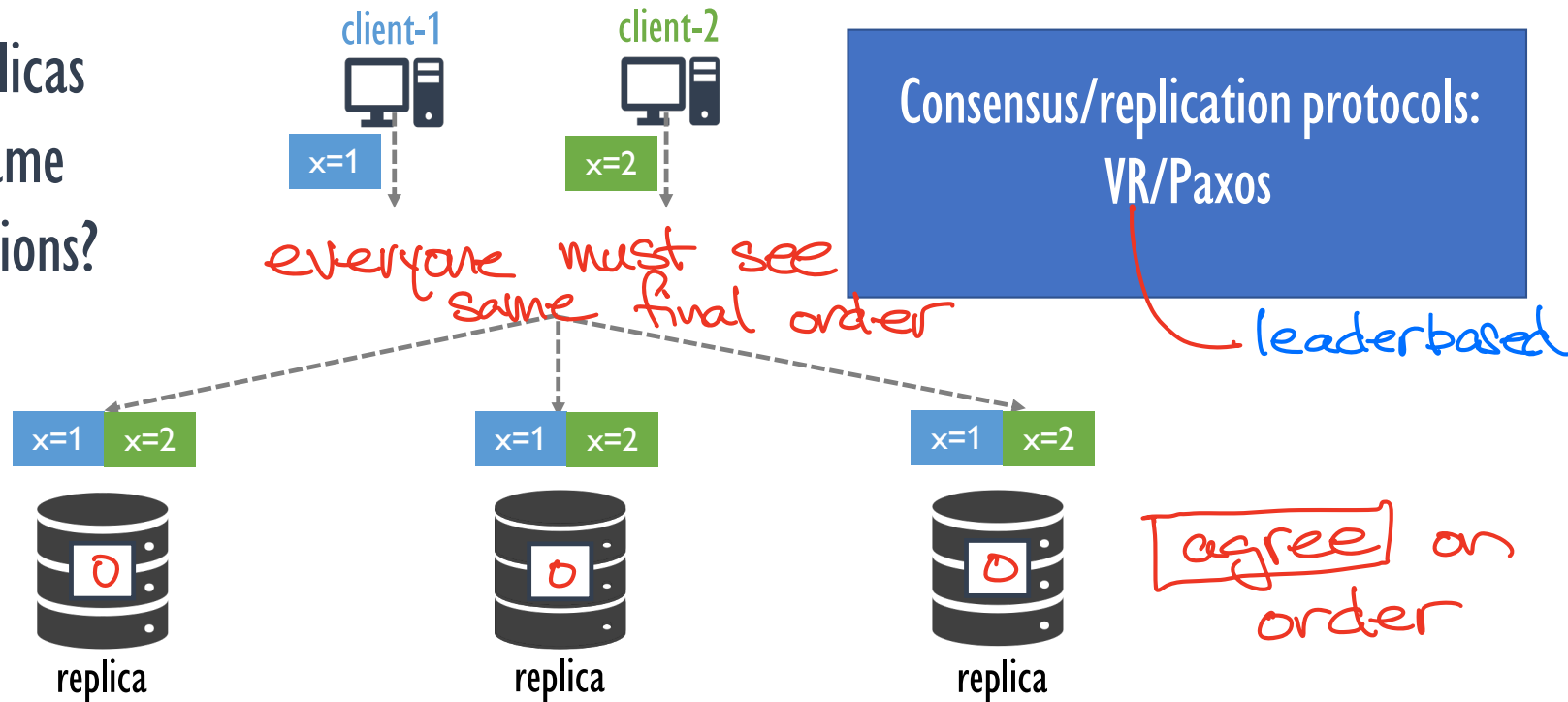
(a $\rightarrow$ b) [Herlihy and Wing 1990]

Reads return



Background: How to build strongly consistent storage?

How to ensure replicas
operate on the same
sequence of operations?



Replicas execute **operations in same order** — state machine replication [Lamport 1978] [Schneider 1982]

real-time order

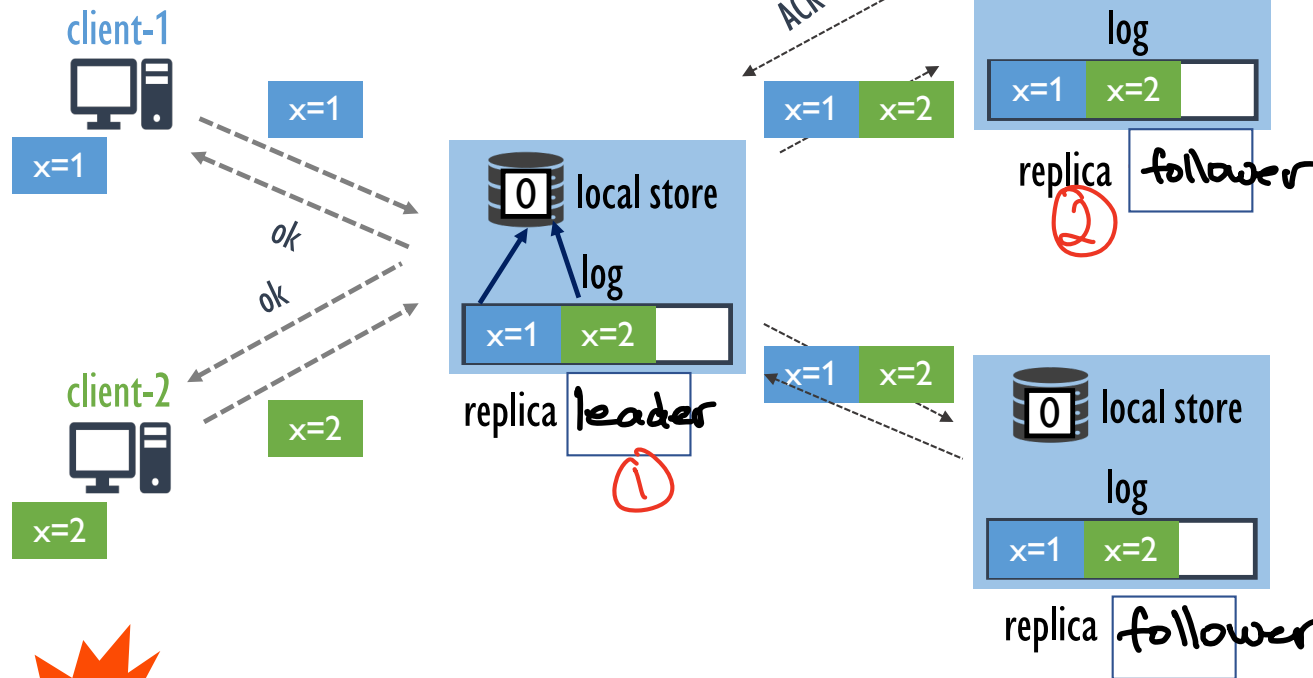
Details important so we can figure out what to defer

Background: Strongly Consistent Replication

⑤

How many RTTs?

common-case
no failures



③

durability: update will not be lost
once **majority** ack

ordering: order agreed by replicas

execution: apply updates to store



④

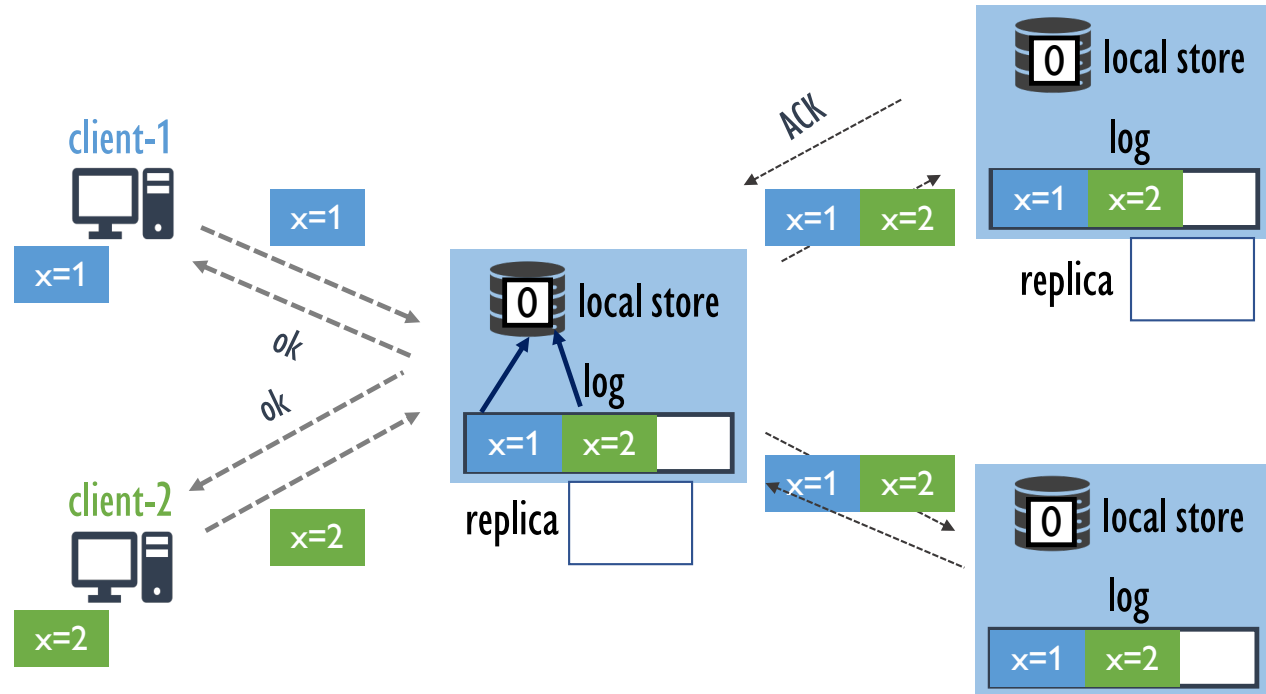
tolerate f failures with $2f+1$ replicas (e.g., 1 failure with three replicas)

Strong Consistency is Expensive

Two RTTs

What other cost?

How is that cost
changing w/ technology?

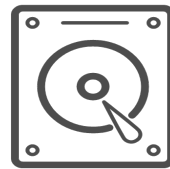


Strong Consistency is Expensive

Two RTTs

Durably persisting updates to disk

Persistence getting cheaper



~10ms



~100us



<1us

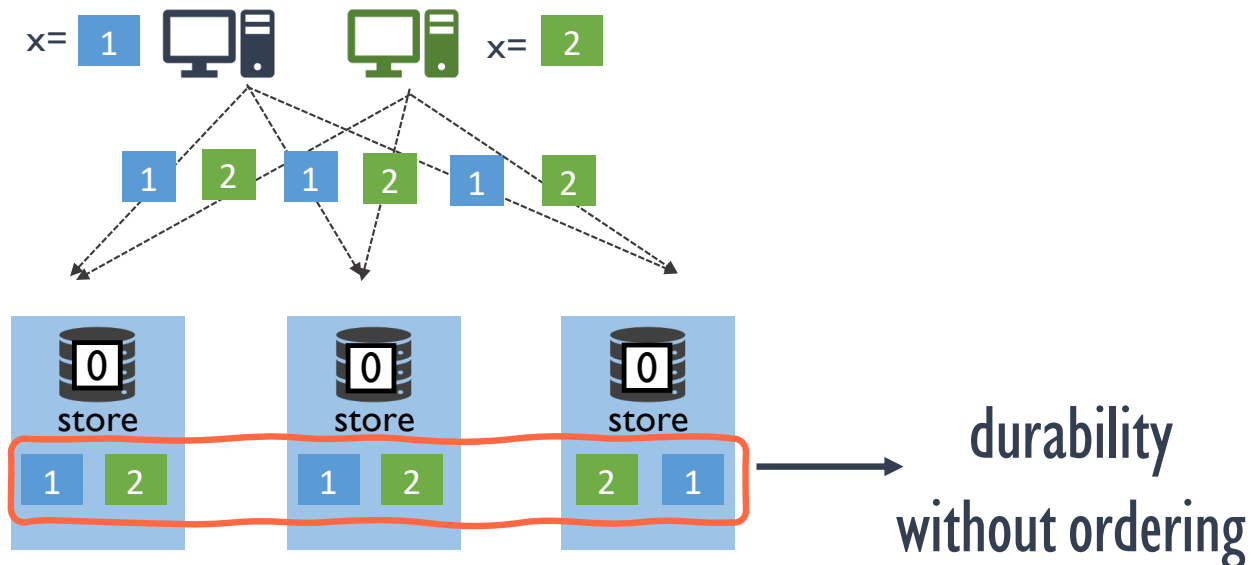
Multiple network roundtrips critical for application performance

high wide-area network latencies

Idea: Durability without Coordination

Durability essential for strong consistency
cannot lose data

Can be achieved without coordination *→ postpone that off critical path*
clients send directly to replicas
How many RTTs for durability?



Ordering requires coordination (incurs 2RTTs)

Can we reduce the ordering cost?

Are there any fundamental opportunities to
avoid ordering in storage **interfaces**?

Outline

Nilext-aware replication

Strongly consistent systems background

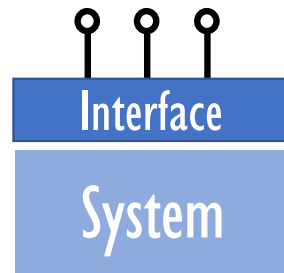
Storage interfaces and nil-externality

Exploiting nil-externality

Skyros design and results



What interface characteristics can make systems “better”?





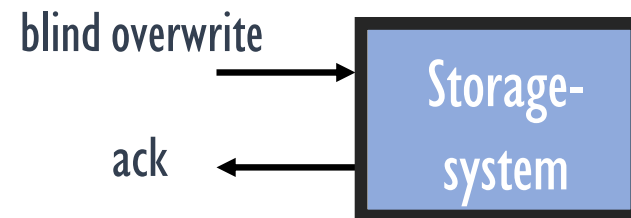
A new interface property — **nil-externality**

Nil-externalizing (or **nilext**) interface

modifies storage system state in any way

does not externalize its effects or system state

no execution result or error



When an interface is **nilext**
a system can **defer** executing the operation → higher **performance**

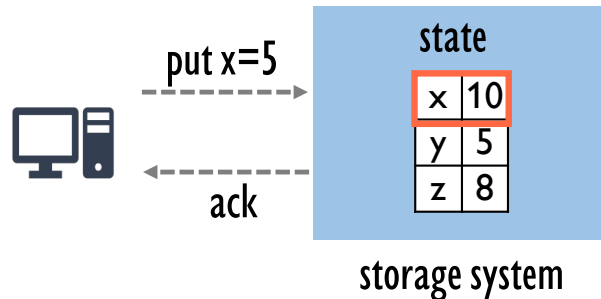
Examples of Nilext vs. Non-nilext

Put interface in KV API — **nilext**

Does **not** return execution **error**

e.g., by checking if key is already present

Does **not** return execution **result** (only ack)



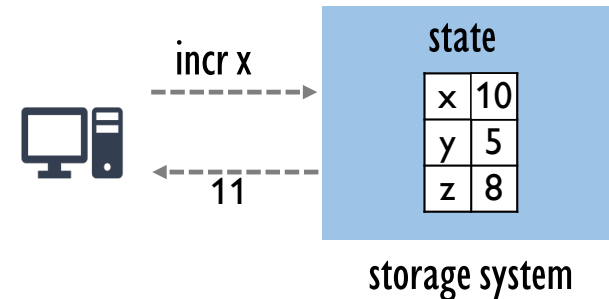
Non-nilext interfaces

Return execution **result**

e.g., incr and return x

Return execution **error**

e.g., get non-existent item



Are there any errors that a nilext function could return?

Nilext Interfaces in Key-Value Stores

Key-value stores (e.g., RocksDB, LevelDB) built upon **write-optimized** structures such as LSM and B^e-trees

Interface	Nilext?	
Put, Write(multi-put)	☐	No error if key(s) already present
Delete	☐	No error if key absent – insert tombstone
Merge (RMW)	☐	Not applied immediately – insert message specifying how to modify value
Get	☐	Returns value or error

Nilext Interfaces are Prevalent

System	Updates		Reads
	Nilext	Non-nilext	
RocksDB	put, write, delete, merge		get, multiget
LevelDB	put, write, delete		get, multiget
Memcached	set	add ^e , delete ^e , cas ^r , replace ^e , append ^e , decr ^e , incr ^r , prepend ^e	get, gets

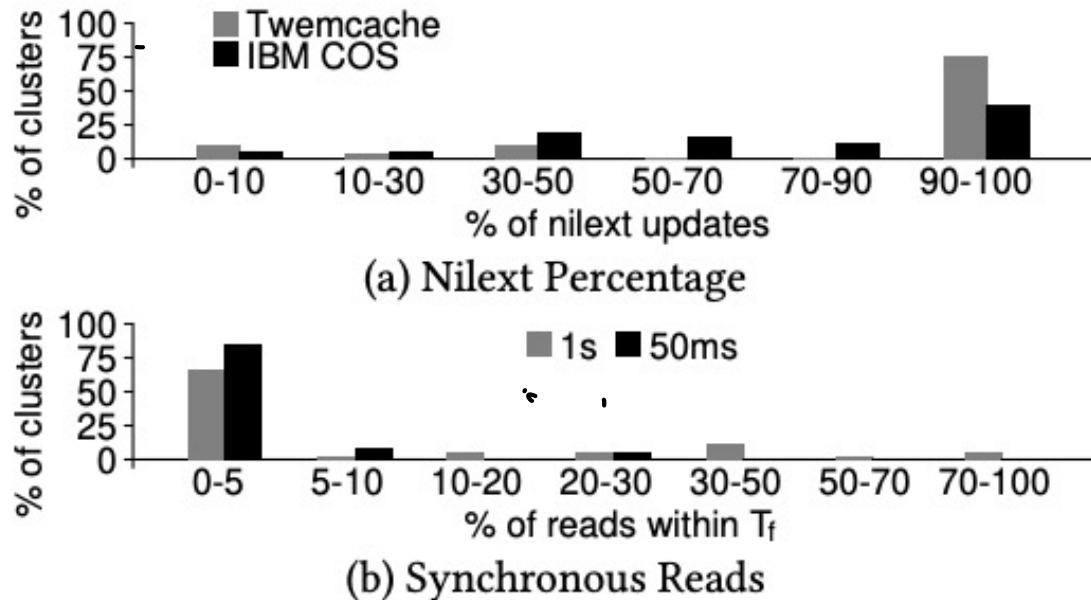
X^e : X is non-nilext because it returns an execution **error** X^r : returns an execution **result**

Some systems have nilext and non-nilext interfaces (e.g., Memcached)
Implication for interface design?

Traces from many clusters, very different usage + operations

Interpretation?

Real-world traces show most updates are nilext



Twemcache:
nilext set-s account for >90% updates in 80% clusters

IBM cloud-object storage traces:
nilext updates account for a vast majority updates

Figure 3. Fast Case is Common. (a) shows the distribution of nilext percentages; a bar for a range $x\%-y\%$ shows the percentage of clusters where $x\%-y\%$ of updates are nilext. (b) shows the distribution of percentage of reads within T_f ; a bar for $x\%-y\%$ shows the percentage of clusters where $x\%-y\%$ of reads access objects updated within T_f . We consider $T_f = 1s, 50ms$.

For clusters with at least
10% updates

Outline

Nilext-aware replication

Strongly consistent systems background

Storage interfaces and nil-externality

Exploiting nil-externality

Skyros design and results



Problem: ordering is expensive (2RTTs)

Key Insight:

Nilext interfaces do not expose state

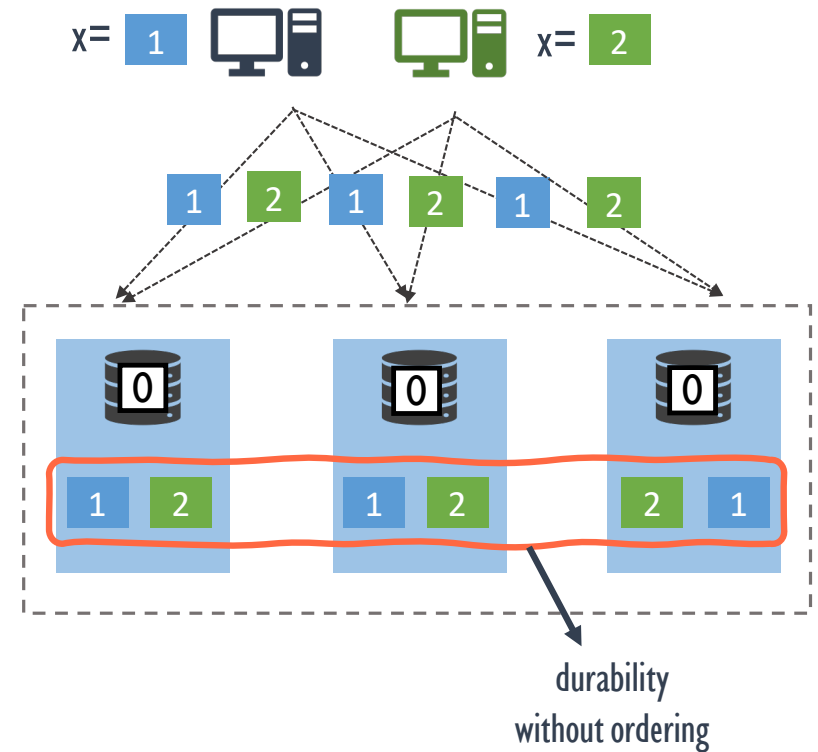
Defer ordering and execution if nilext → performance (1RTT)
without affecting consistency

New Idea: Exploiting Nil-Externality

1 Durability without coordination

2

3



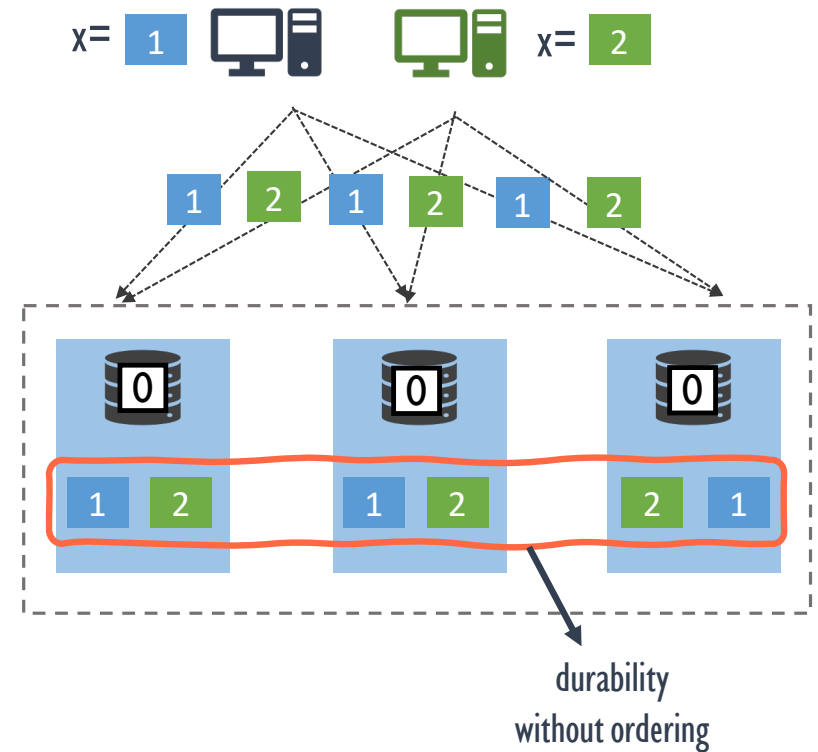
New Idea: Exploiting Nil-Externality

- 1 Durability without coordination
- 2 Interface nilext → defer ordering (and execution)

nilext update does not externalize state

1 RTT completion — low latency

- 3



New Idea: Exploiting Nil-Externality

- 1 Durability without coordination
- 2 Interface nilext → defer ordering (and execution)

nilext update does not externalize state

1 RTT completion — low latency

- 3 Non-nilext operations externalize state

enforce ordering and execution before state externalized —

strong consistency

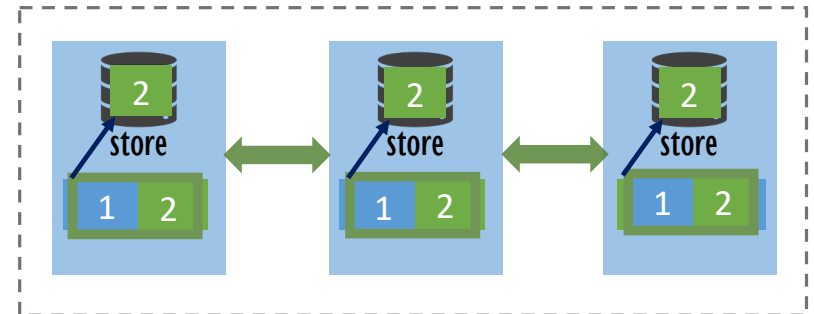
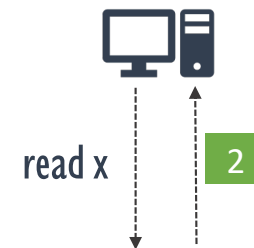
common case: updates already ordered and executed

done in background

Performance

no coordination in critical path — low latency

batching requests — high throughput



Strong consistency

inconsistency not exposed as interface nilext

enforce consistency before externalizing

New Idea: Exploiting Nil-Externality

3 Non-nilext operations externalize state

enforce ordering and execution before state externalized —

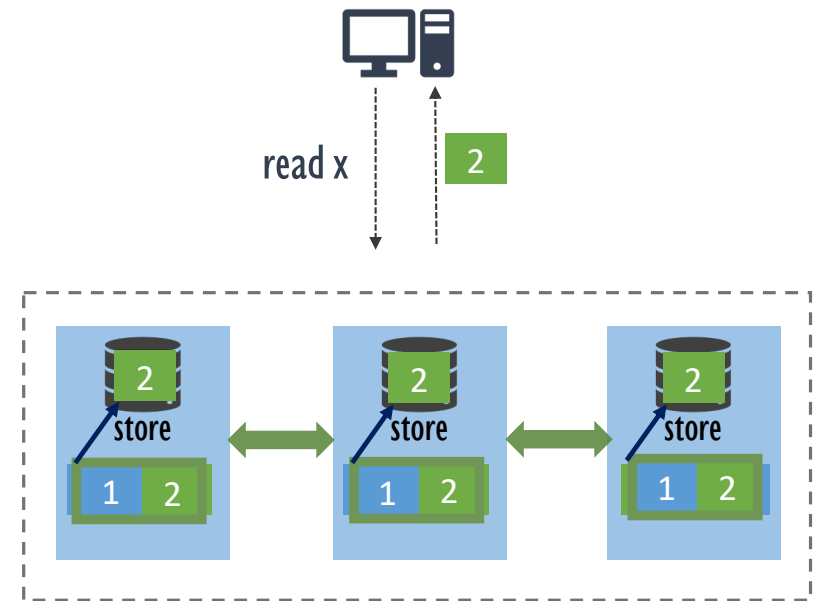
strong consistency

common case: updates already ordered and executed
done in background

Provide linearizability?

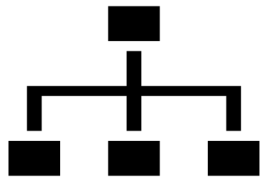
Do all clients see same order?

Does reading “x” cause all older ops to be executed or just “last write to x”?



Strong consistency
inconsistency **not exposed** as interface nilext
enforce consistency before **externalizing**

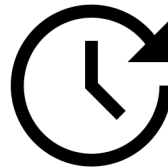
Other Approaches



Network Ordering

[Dang et al., 2015] [Li et al., 2016]

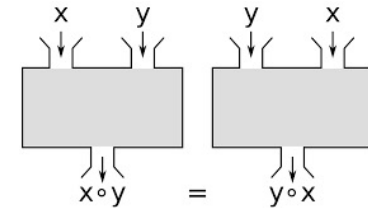
special hardware
not for geo-replication



Speculative Execution

[Kemmer et al., 1999] [Kotla et al., 2007] [Ports et al., 2015]

mis-speculation cost
affects tail



Exploiting Commutativity

[Lamport 2005] [Moraru et al., 2013] [Park et al., 2019]

expensive when interfaces
do not commute

Existing protocols **always eagerly** order/execute

Contribution: identify a new opportunity in storage **interfaces**
exploit nil-externality — **defer** ordering and execution

works for geo-
replication

no expensive rollbacks
improves the tail

nil-externality
+ commutativity

Outline

Nilext-aware replication

Strongly consistent systems background

Storage interfaces and nil-externality

Exploiting nil-externality

Skyros design and results



Skyros

Skyros — new nilext-aware replication protocol

Leader-based

Guarantees:

availability — tolerate up to f failures in a system with $2f+1$ replicas

strong consistency (linearizability)

Skyros Overview

Nilext updates: clients write to replicas directly

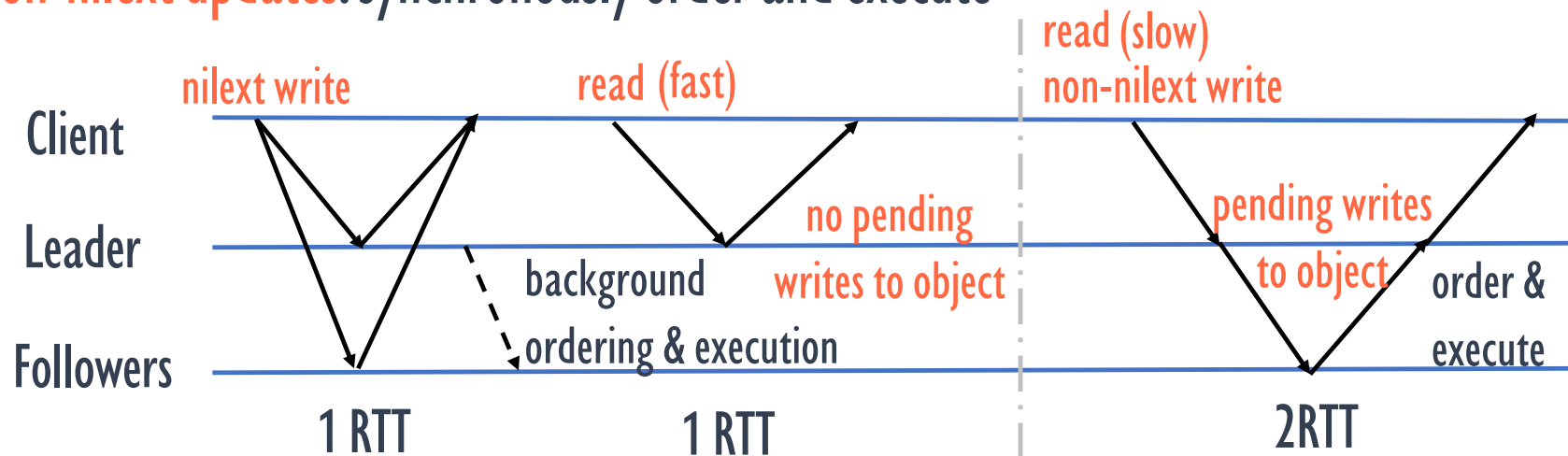
wait for ack from **supermajority** ($f + \lceil f/2 \rceil + 1$) **including leader** (4 replicas in 5-node cluster)

Reads: at leader

mostly 1 RTT — no unordered updates for object

sometimes 2 RTTs — when they access not-yet ordered updates

Non-nilext updates: synchronously order and execute



Performance — Fast Case is the Common Case

✓ Nioxt interfaces **prevalent** in many storage systems
frequently used by applications

✓ Reads do **not** often **access recent** updates

real-world traces — IBM COS

85% of clusters, <5% of reads access recently modified objects

similar trends in popular cloud storage benchmarks

Correctness — Common Case

Invariant — leader has all updates in **real-time order**

Reads and non-nilext updates at leader
will see effects of nilext updates

→ **strong consistency**

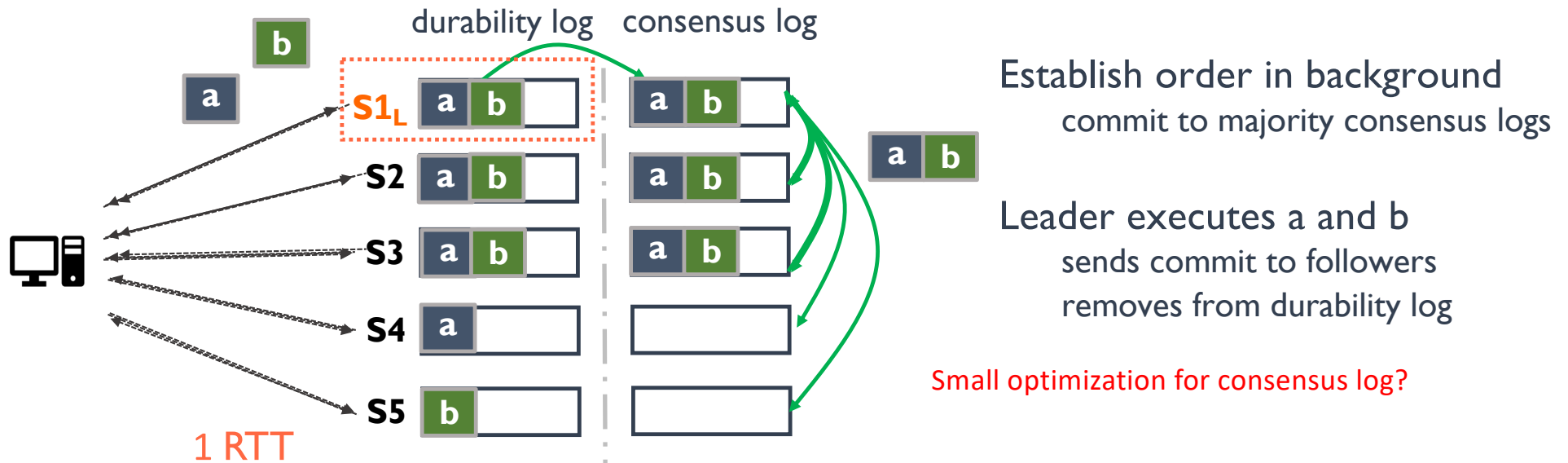
Nilext Updates in Skyros

Durability log in addition to the usual consensus log

Clients write to replicas directly

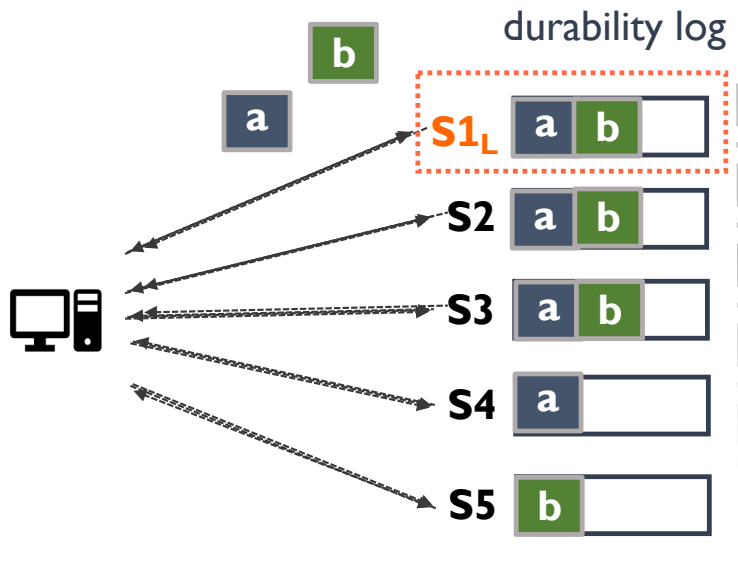
Wait for ack from supermajority ($f + \lceil f/2 \rceil + 1$) **including leader** (4 replicas in 5-node cluster)

Leader's durability log has updates in correct order ($a \rightarrow b$ in real time; log reflects that)



The hard part — what if the leader fails?

New leader must recover all completed updates in correct order



1. all completed operations **durable**
2. **real-time order** preserved
3. once ordered, order survives

reason for
supermajority

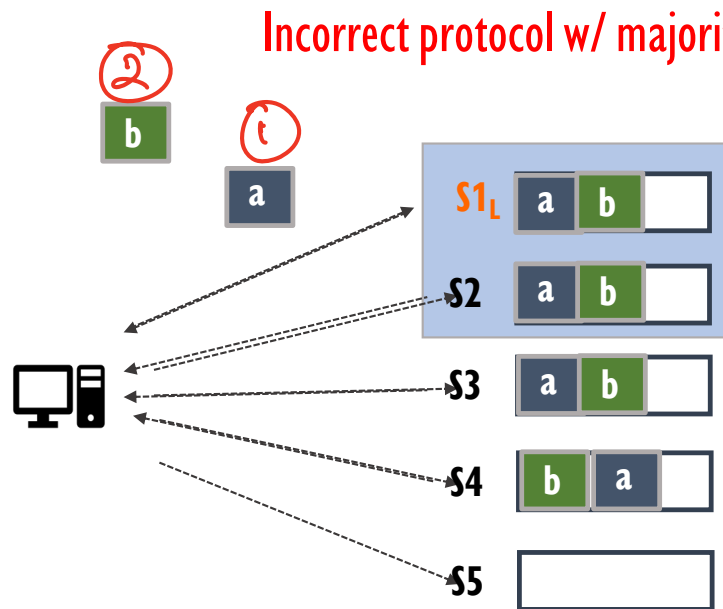
Why Supermajority?

Why not $f+1$ (majority)?

Example: $f=2$, majority = 3, supermajority=4

$a \rightarrow b$ in real time

$$f + \lceil f/2 \rceil + 1$$



*always
include
leader*

System must be available after f failures

S3: a b S4: b a S5:

Which ordering is correct?

Intuition: $f + \lceil f/2 \rceil + 1$ ensures a **majority in $f+1$** has correct order

Reconstruct linearizable order using
DAG-based order-resolution technique

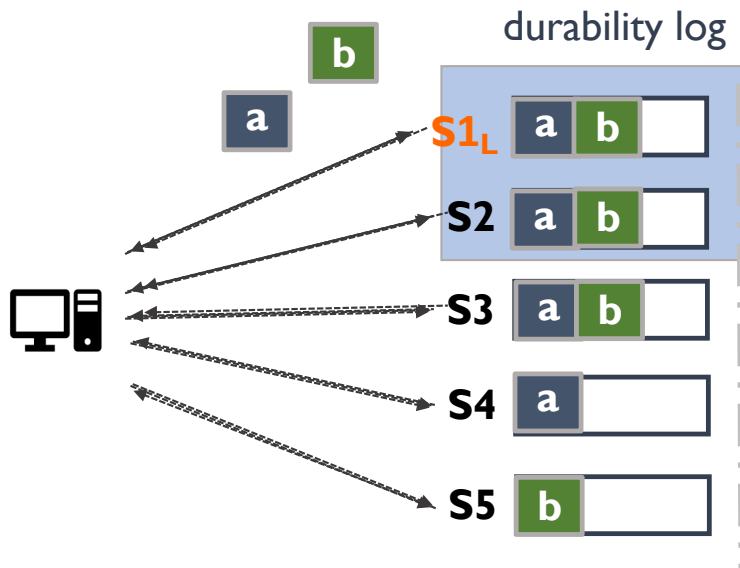
Nilext Updates in Skyros

Durability log in addition to the usual consensus log

Clients write to replicas directly

Wait for ack from supermajority ($f + \lceil f/2 \rceil + 1$) including leader (4 replicas in 5-node cluster)

Correct protocol w/ super majority



System must be available after f failures

S3: a b S4: a S5: b

Intuition: $f + \lceil f/2 \rceil + 1$ ensures a **majority in $f+1$** has correct order

Reconstruct linearizable order using DAG-based approach

exact procedure — not in this talk...

View Changes

What if current leader fails?

new leader must recover the durability log

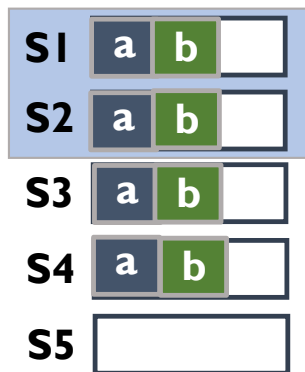
recover all completed updates in correct order

Use durability logs from $f+1$ replicas (including new leader)

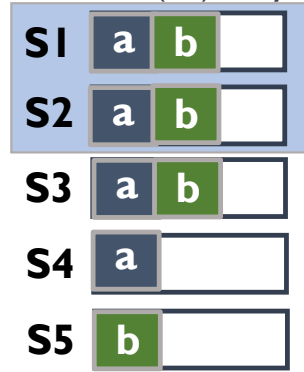
Intuition: for any two operations, a and b

b follows a in real time

supermajority logs have
a then b



supermajority logs have
a then b (or) only a



a majority within any available $f+1$ will have

a then b (or) only a

if a and b are **concurrent**

leader establishes some order

once established, order survives failures

Reconstruct linearizable order using
DAG-based order-resolution technique

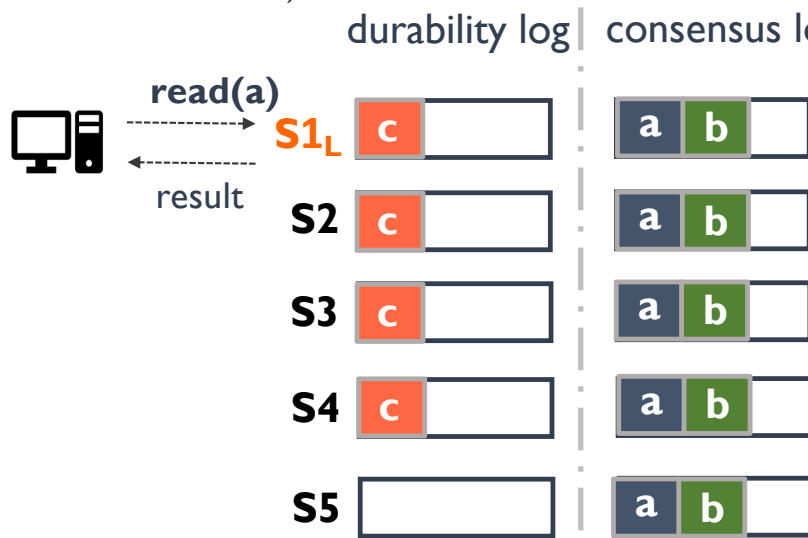
Please see paper ...

Reads in Skyros

Reads to leader

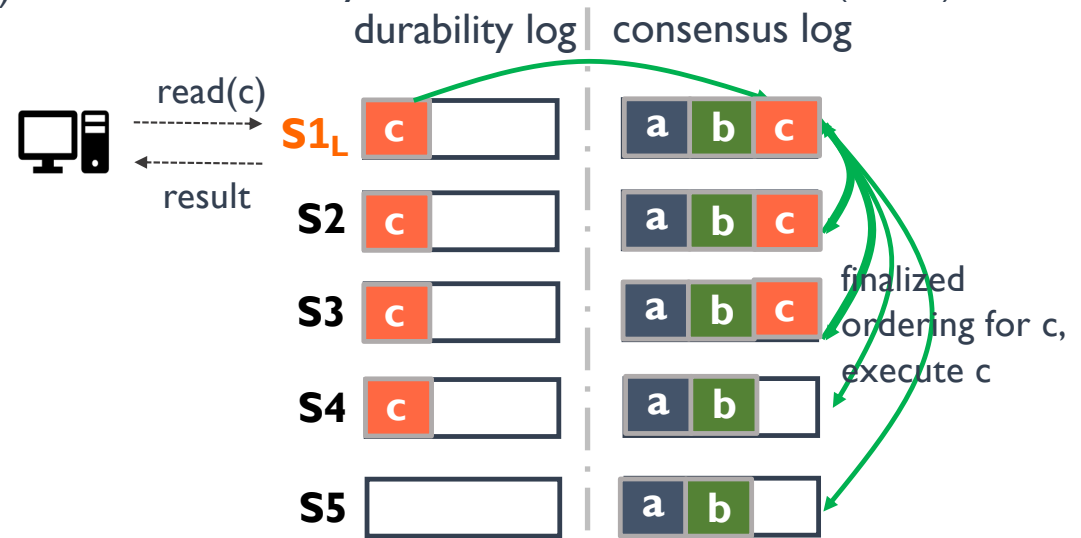
Leader checks durability log to see if there are unordered updates for object

If no, leader returns result of read (1RTT)



a already ordered and executed

If yes, order and execute now (2RTT)



c not yet ordered

Leader synchronously orders and executes **non-nilext updates**

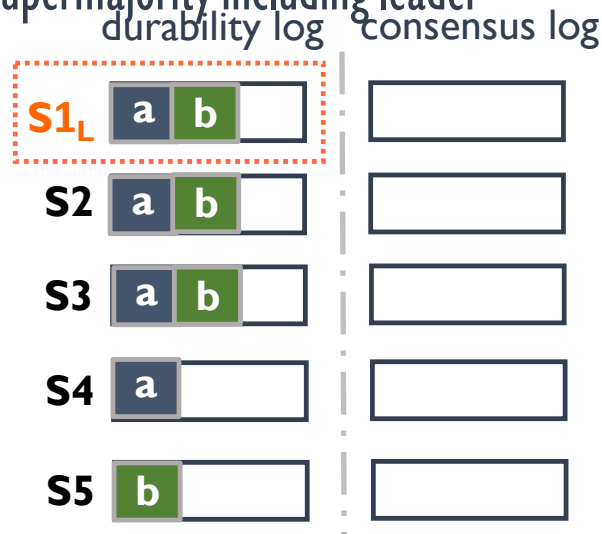
Correctness: Common Case

Leader's durability log contains all completed updates in correct order

Operations a and b; b follows a in real time

a will appear before b in leader's durability log

updates acknowledged by supermajority including leader



Reads and non-nilext updates go to leader; thus, will see effects of nilext updates

Skyros Implementation

Implemented the protocol and built a KV store

Correctness proof in paper

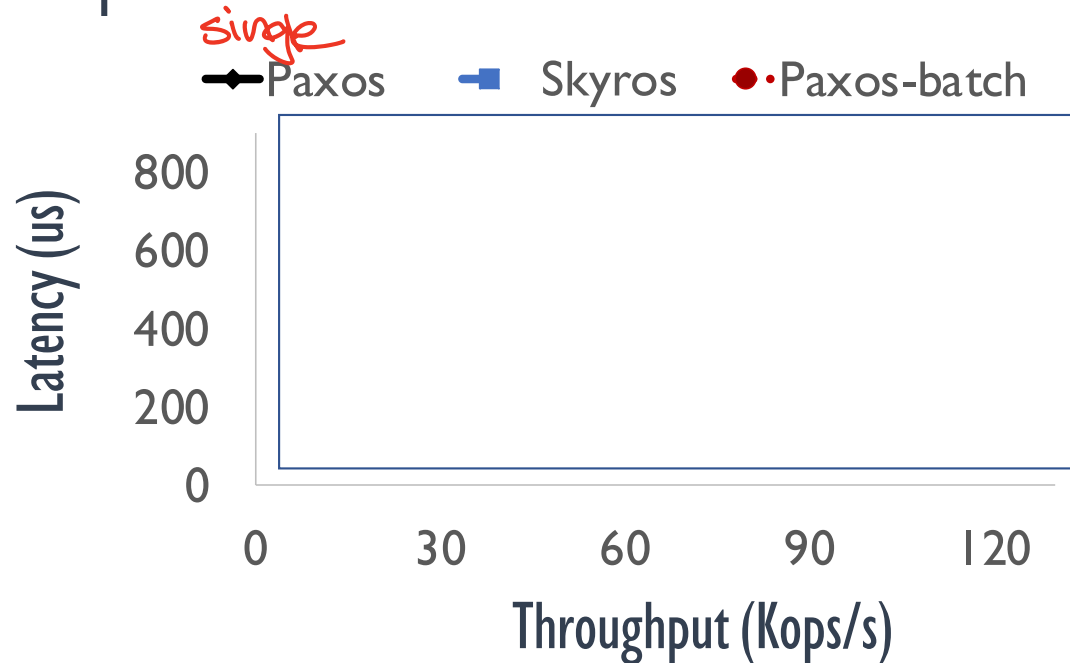
Extensive benchmarks to demonstrate benefits

in today's talk — just a couple of results

What are the Benefits of Exploiting Nil-Externality?

Workload: nil-ext-only updates; vary number of clients

Comparison with Paxos variants



If everything is nil-ext

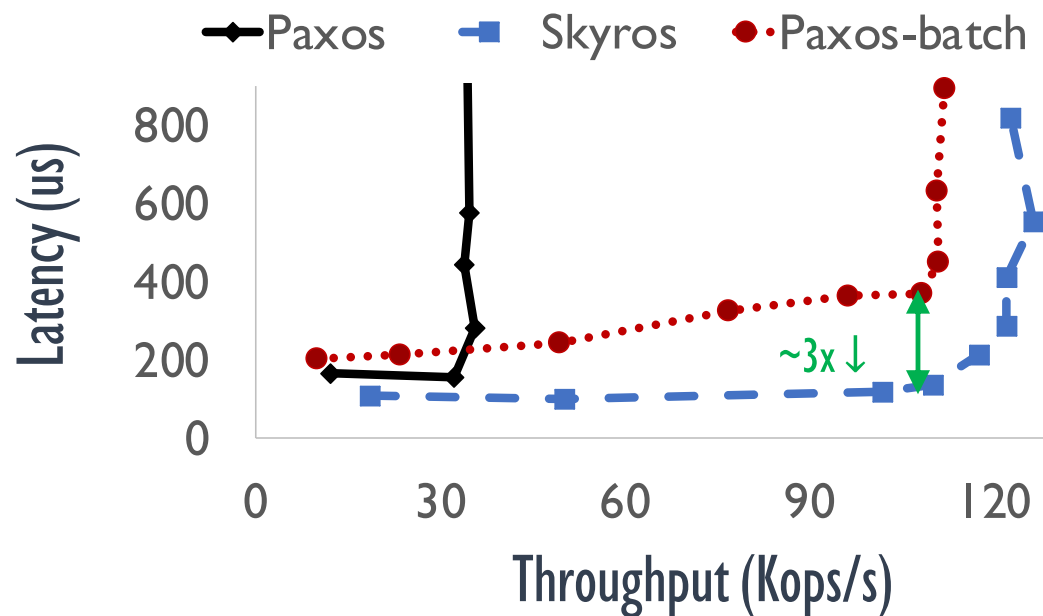
best case for Skyros

What do you expect?

What are the Benefits of Exploiting Nil-Externality?

Workload: nilext-only updates; vary number of clients

Comparison with Paxos variants



up to 3.4x throughput improvements over Paxos (without batching)

significant reduction in latency over Paxos w/ batching

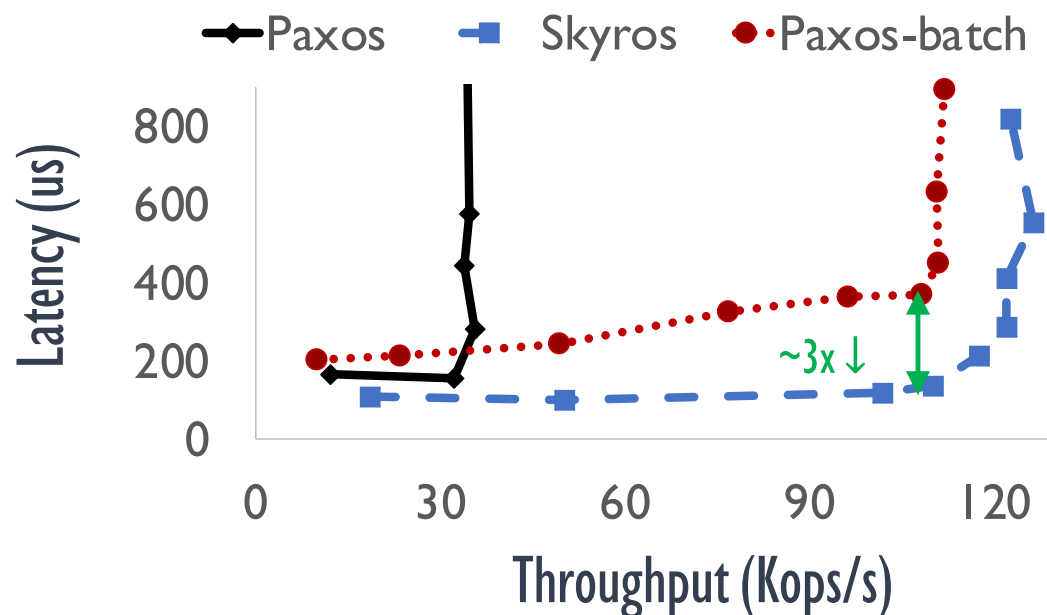
Why better throughput?

better latency: 1RTT

What are the Benefits of Exploiting Nil-Externality?

Workload: nilext-only updates; vary number of clients

Comparison with Paxos variants



up to 3.4x throughput improvements over Paxos (without batching)

significant reduction in latency over Paxos w/ batching

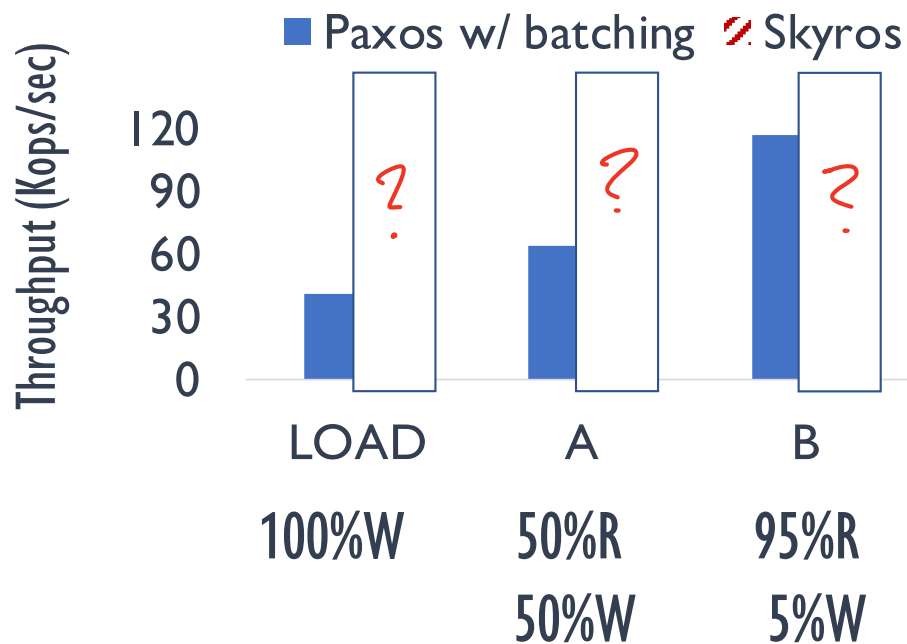
Why better throughput?

Can defer work until idle time

How does Skyros Perform on Mixed Workloads?

Use YCSB, a benchmark for cloud storage systems [Cooper et al., 2010]

mix of nilext writes and reads under fixed clients



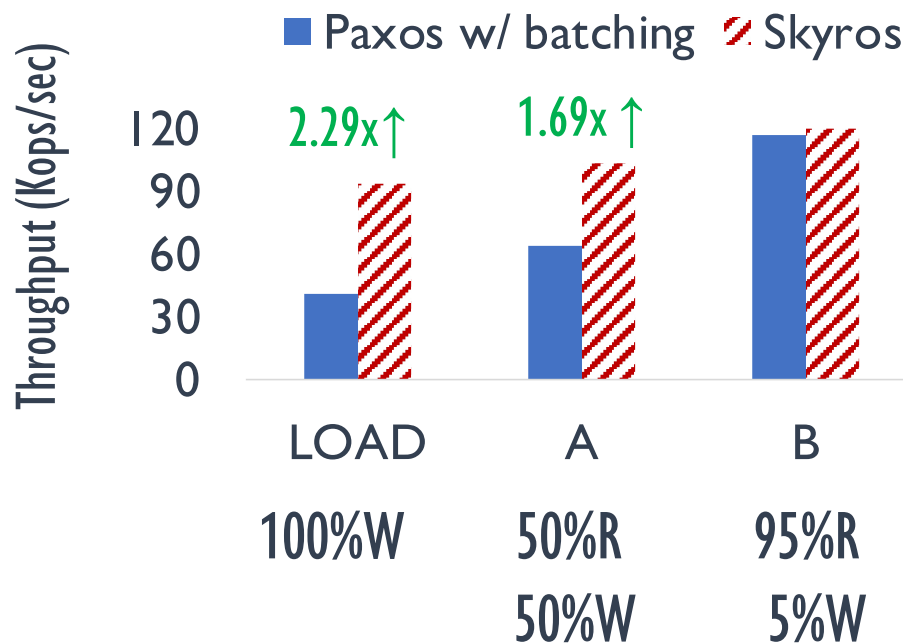
More benefits with Skyros for
read or write heavy workloads?

Paxos: reads are faster
than writes

How does Skyros Perform on Mixed Workloads?

Use YCSB, a benchmark for cloud storage systems [Cooper et al., 2010]

mix of nilext writes and reads under fixed clients



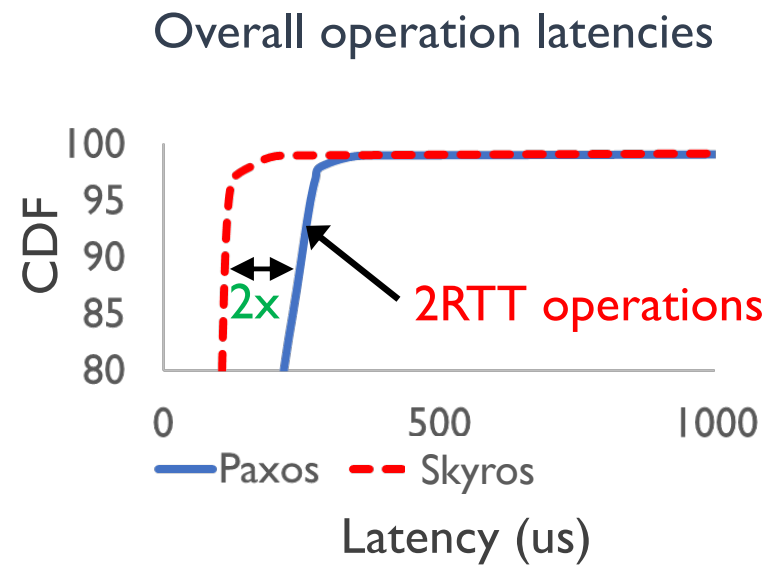
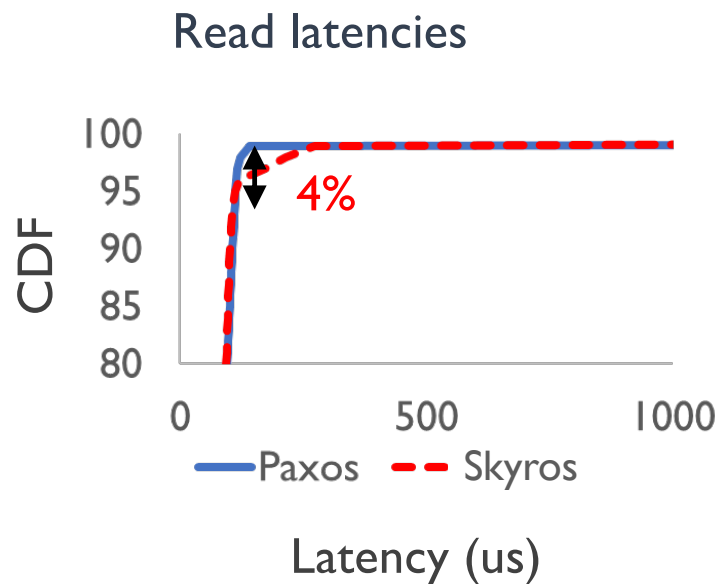
Significant benefits for write-heavy workloads

For read-heavy workloads, Skyros reduces
p99 latency by 70% (not shown here) *- next slides*
all ops

Only 0.3% — 2% operations take 2RTT in Skyros

Does Skyros Incur Overheads on Reads?

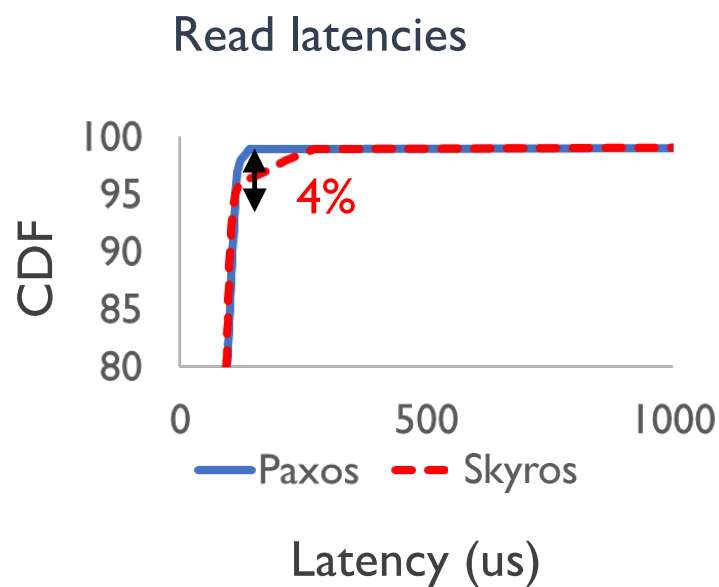
Workload: YCSB-A: 50% writes and 50% reads



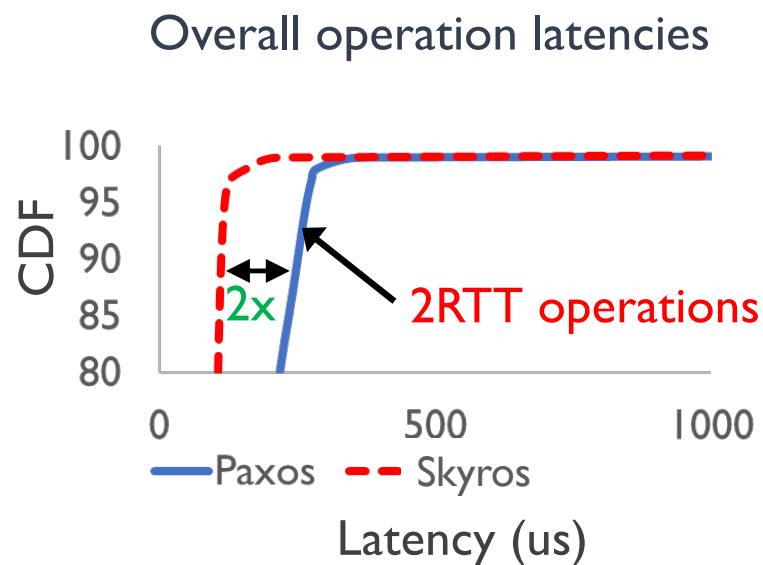
Observations?
(Figure 11(B) and 11(C) in paper)

Does Skyros Incur Overheads on Reads?

Workload: YCSB-A: 50% writes and 50% reads



Most read operations in Skyros are fast
Only 4% reads slow due to synchronous ops



Slow reads do not affect overall workload tail
in fact, improves tail (p99 by 70%)

Impact of Recency

how quickly after?

Three Observations?

50% nilent writes and 50% reads with 10 clients

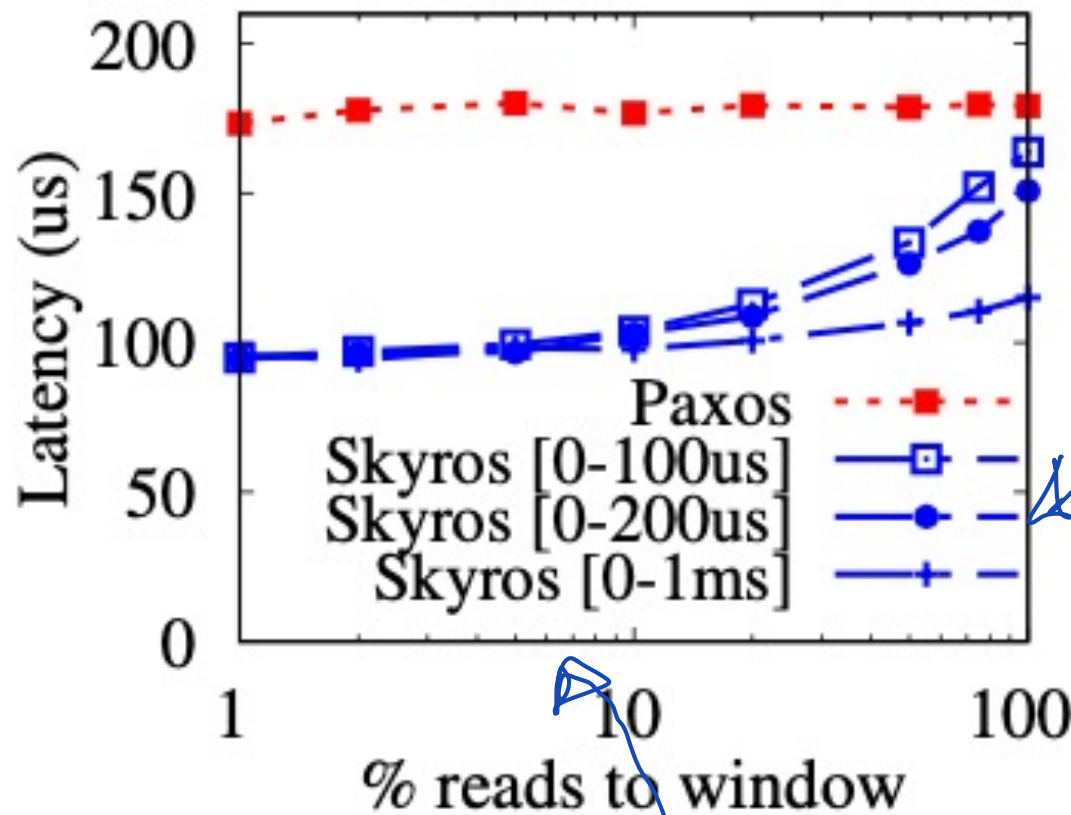
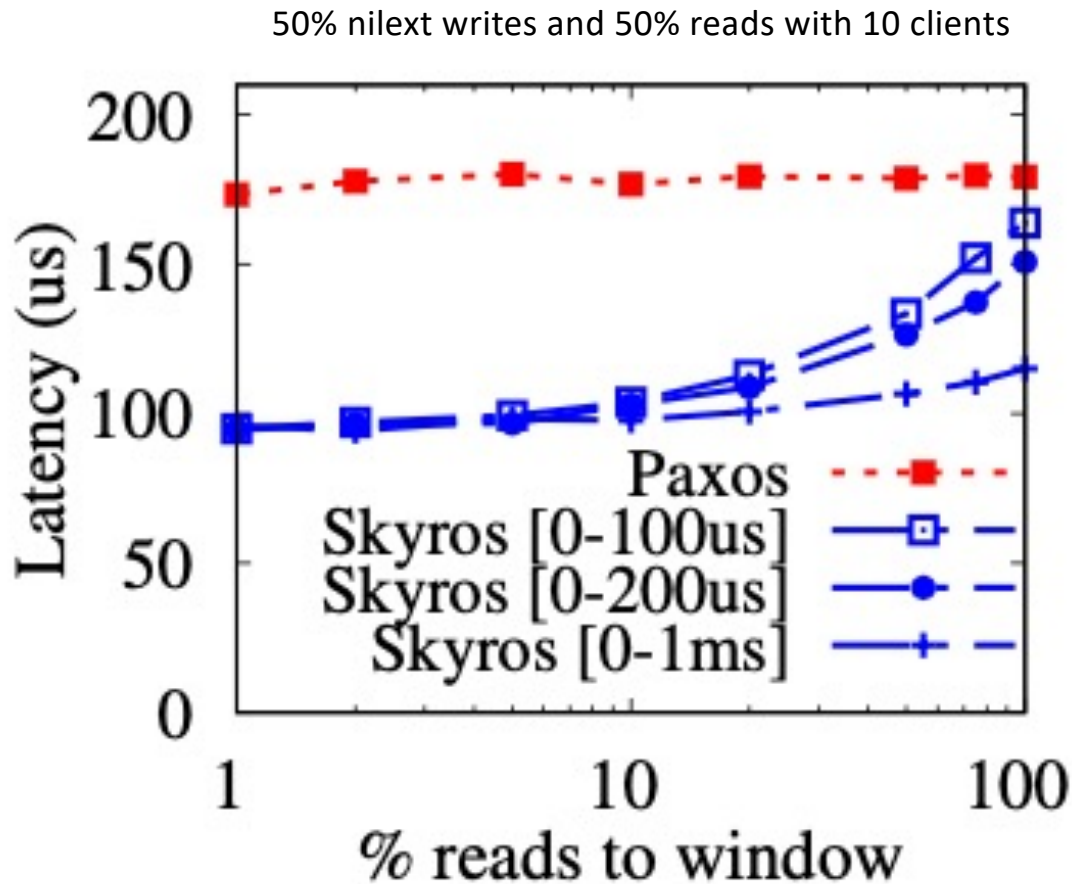


Fig. 9 in paper

how many are quick reads?

Impact of Recency



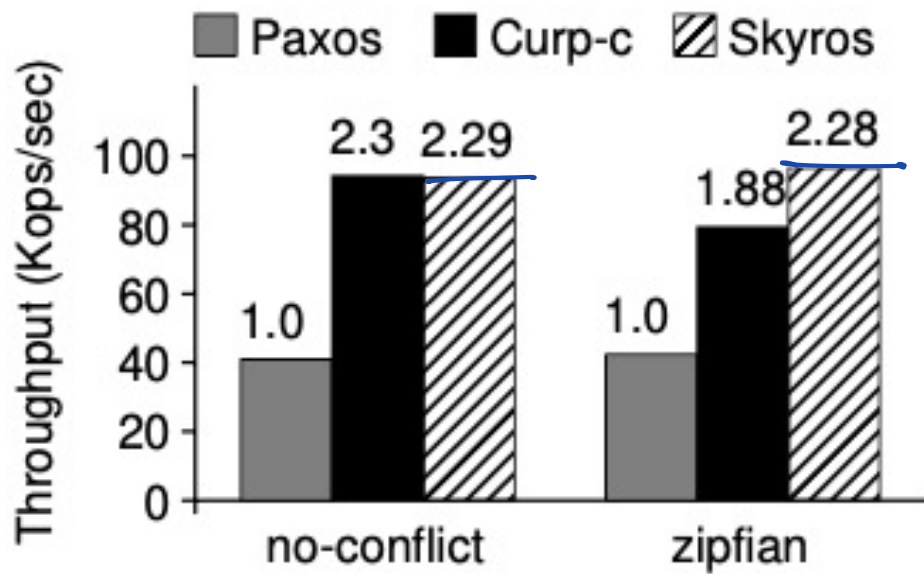
Three Observations?

Performance of Paxos is not sensitive to recency of reads

Skyros has lower latency when most reads are to "old data"

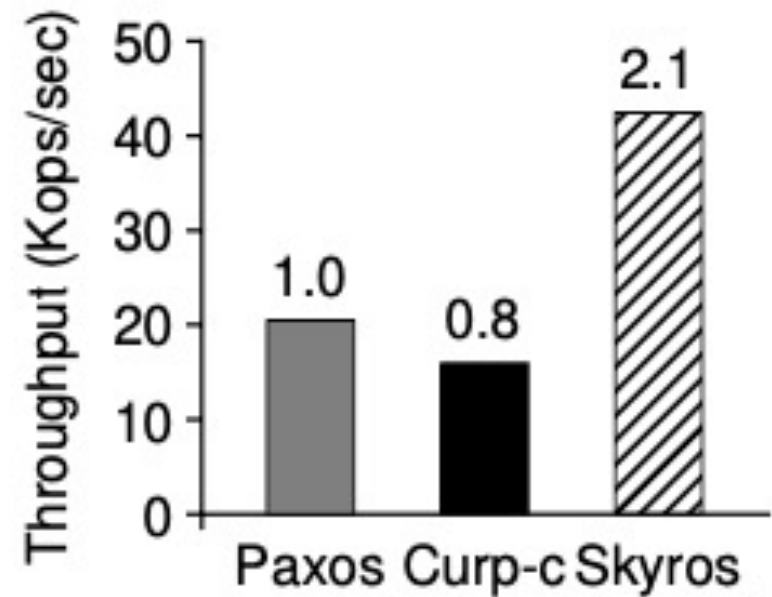
Skyros has higher latency when reads to recently-written data are very recent (e.g., <100us instead of < 1ms)

Compare to Commutativity (EPaxos)



all commutative

(a) Key-value write-only - nilext



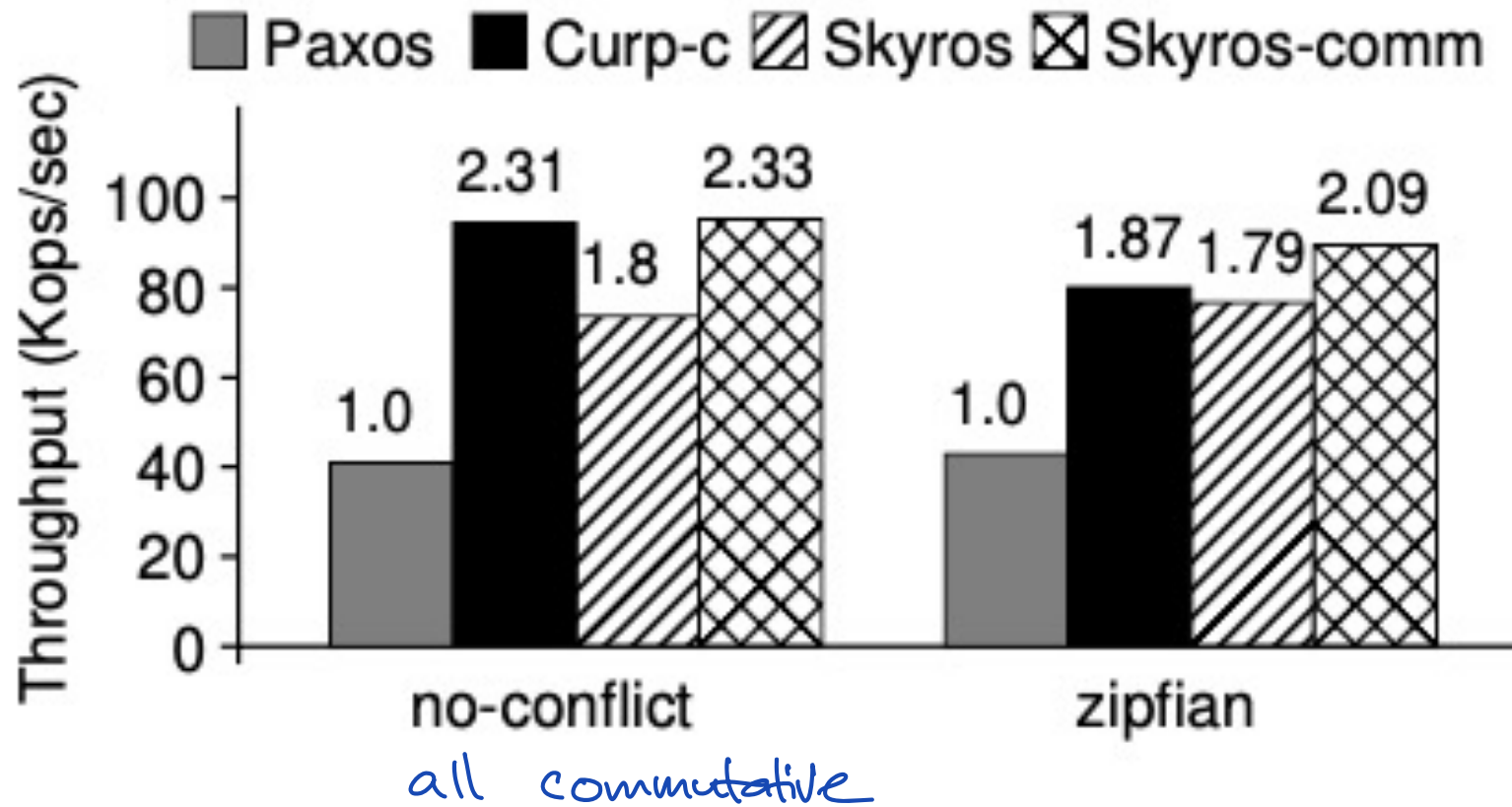
(d) Record append

Skyros performs same
as all commutative

- doesn't require ops to
be commutative

Combine with Commutativity

10% non-nilext writes, write-only



Nilext-aware Replication Summary

Strong consistency is expensive (incurs 2RTTs)

Skyros — a new **replication protocol** that provides strong consistency and incurs mostly **1RTT**

- built a practical system

- 3.1x lower latencies, better tail

- up to 2.3x better throughput

Contributions:

- Identify **nil-externality** — a **storage interface property**

- A new protocol that exploits nil-externality — defer ordering and execution